

KUBERNETES_TASK_3

Todays_Task:

1. Create a ClusterIP service for an Apache web server pod.
2. Expose an Nginx pod externally using a NodePort service.
3. Deploy a ReplicationController to maintain 3 replicas of an Nginx pod.
4. Scale the ReplicationController from 3 replicas to 5 replicas.
5. Create a ReplicaSet to manage pods based on multiple labels (prod and test).
6. Deploy a ReplicaSet that excludes pods with the label backend.
7. Test load balancing across multiple pods using a NodePort service.
8. Delete a ReplicationController without affecting the running pods.

Summary

In this document, I am going to practice Kubernetes core concepts to strengthen my understanding of Pods, Containers, Services, ReplicationController, and ReplicaSet. I will perform each task step by step using YAML files and kubectl commands, and I will verify the output using commands like kubectl get, kubectl describe, kubectl logs, kubectl exec, and kubectl get endpoints. This practice will help me understand how Kubernetes resources are created, managed, exposed, and scaled in a real environment. These notes are being prepared as a reference for future revision and interview preparation.

Task 1: Create a ClusterIP service for an Apache web server pod

Step 1: Create Apache pod YAML

Create file:

```
vi apache-pod.yaml
```

Add:

```
apiVersion: v1
```

```
kind: Pod
```

metadata:

name: apache-pod

labels:

app: apache

spec:

containers:

- name: apache

image: httpd

ports:

- containerPort: 80

Explanation: Pod is created with label app=apache so service can select it.

```
root@master:~# vi apache-pod.yaml
root@master:~# cat apache-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: apache-pod
  labels:
    app: apache
spec:
  containers:
    - name: apache
      image: httpd
      ports:
        - containerPort: 80
```

Step 2: Apply pod

kubectl apply -f apache-pod.yaml

Step 3: Verify pod

kubectl get pods

```
root@master:~# kubectl apply -f apache-pod.yaml
pod/apache-pod created
root@master:~# kubectl get pods
NAME          READY   STATUS    RESTARTS   AGE
apache-pod    1/1     Running   0           12s
root@master:~#
```

Step 4: Create ClusterIP service YAML

Create file:

vi apache-svc.yaml

Add:

apiVersion: v1

kind: Service

metadata:

name: apache-service

spec:

type: ClusterIP

selector:

app: apache

ports:

- port: 80

targetPort: 80

Explanation: ClusterIP gives internal access inside cluster only.

```
root@master:~# vi apache-svc.yaml
root@master:~# cat apache-svc.yaml
apiVersion: v1
kind: Service
metadata:
  name: apache-service
spec:
  type: ClusterIP
  selector:
    app: apache
  ports:
    - port: 80
      targetPort: 80
```

Step 5: Apply service

kubectl apply -f apache-svc.yaml

Step 6: Verify service

kubectl get svc

```
root@master:~# kubectl apply -f apache-svc.yaml
service/apache-service created
root@master:~# kubectl get svc
NAME                TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)    AGE
apache-service      ClusterIP   10.99.14.121   <none>         80/TCP     31s
kubernetes           ClusterIP   10.96.0.1      <none>         443/TCP    8m31s
root@master:~#
```

Step 7: Test service from inside pod

➔ Verify

“kubectl describe svc apache-service” and match endpoint with pod ip

```

root@master:~# kubectl apply -f apache-svc.yaml
service/apache-service created
root@master:~# kubectl get svc
NAME                TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)    AGE
apache-service      ClusterIP   10.99.14.121   <none>         80/TCP     31s
kubernetes           ClusterIP   10.96.0.1      <none>         443/TCP    8m31s
root@master:~# kubectl describe svc apache-service
Name:                apache-service
Namespace:           default
Labels:              <none>
Annotations:         <none>
Selector:             app=apache
Type:                ClusterIP
IP Family Policy:    SingleStack
IP Families:         IPv4
IP:                  10.99.14.121
IPs:                 10.99.14.121
Port:                <unset> 80/TCP
TargetPort:          80/TCP
Endpoints:           10.10.226.82:80
Session Affinity:    None
Internal Traffic Policy: Cluster
Events:              <none>
root@master:~# kubectl get pods wide -o
error: flag needs an argument: 'o' in -o
see 'kubectl get --help' for usage
root@master:~# kubectl get pods -o wide
NAME          READY   STATUS    RESTARTS   AGE   IP            NODE      NOMINATED NODE   READINESS GATES
apache-pod    1/1     Running   0           11m   10.10.226.82  worker-1  <none>           <none>
root@master:~#

```

Explanation: Service name works as DNS inside cluster.

Task 2: Expose an Nginx pod externally using NodePort service

Step 1: Create nginx pod YAML

vi nginx-pod.yaml

Add:

apiVersion: v1

kind: Pod

metadata:

name: nginx-pod

labels:

app: nginx

spec:

containers:

- name: nginx

image: nginx

ports:

- containerPort: 80

```
root@master:~  
root@master:~# vi nginx-pod.yaml  
root@master:~# cat nginx-pod.yaml  
apiVersion: v1  
kind: Pod  
metadata:  
  name: nginx-pod  
  labels:  
    app: nginx  
spec:  
  containers:  
  - name: nginx  
    image: nginx  
    ports:  
    - containerPort: 80
```

Explanation: Label app=nginx is needed for NodePort service selector.

Step 2: Apply pod

kubectl apply -f nginx-pod.yaml

```
root@master:~# kubectl apply -f nginx-pod.yaml  
pod/nginx-pod created  
root@master:~# kubectl get pods  
NAME          READY   STATUS    RESTARTS   AGE  
apache-pod    1/1     Running   0           31m  
nginx-pod     1/1     Running   0           11s  
root@master:~# |
```

Step 3: Create NodePort service YAML

vi nginx-nodeport.yaml

Add:

apiVersion: v1

kind: Service

metadata:

name: nginx-nodeport

spec:

type: NodePort

selector:

app: nginx

ports:

- port: 80

targetPort: 80

nodePort: 30080

```

root@master:~# vi nginx-nodeport.yaml
root@master:~# cat nginx-nodeport.yaml
apiVersion: v1
kind: Service
metadata:
  name: nginx-nodeport
spec:
  type: NodePort
  selector:
    app: nginx
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30080

```

Explanation: NodePort exposes service outside cluster using node IP and port.

Step 4: Apply service

```
kubectl apply -f nginx-nodeport.yaml
```

Step 5: Verify service

```
kubectl get svc
```

```

root@master:~# kubectl apply -f nginx-nodeport.yaml
service/nginx-nodeport created
root@master:~# kubectl get svc

```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
apache-service	ClusterIP	10.99.14.121	<none>	80/TCP	31m
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	39m
nginx-nodeport	NodePort	10.106.95.89	<none>	80:30080/TCP	10s

```

root@master:~#

```

Step 6: Get node IP

```
kubectl get nodes -o wide
```

```

root@master:~# kubectl get nodes -o wide

```

NAME	STATUS	ROLES	AGE	VERSION	INTERNAL-IP	EXTERNAL-IP	OS-IMAGE	KERNEL-VERSION	CONTAINER-RUNTIME
master	Ready	control-plane	44h	v1.34.4	172.31.32.135	<none>	Ubuntu 24.04.3 LTS	6.14.0-1018-aws	containerd://1.7.28
worker-1	Ready	<none>	42h	v1.34.4	172.31.46.47	<none>	Ubuntu 24.04.3 LTS	6.14.0-1018-aws	containerd://1.7.28
worker-2	Ready	<none>	42h	v1.34.4	172.31.46.88	<none>	Ubuntu 24.04.3 LTS	6.14.0-1018-aws	containerd://1.7.28

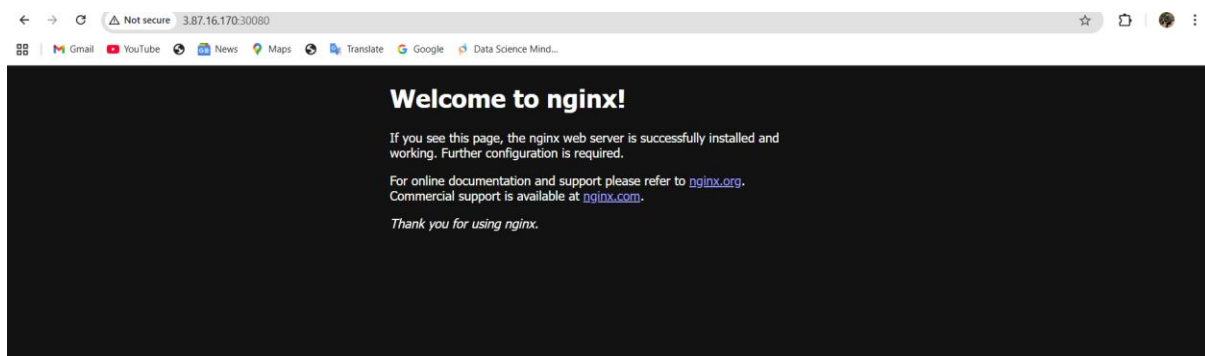
```

root@master:~#

```

Step 7: Access nginx from browser or curl

```
curl http://<NODE-IP>:30080
```



```

worker-2 Ready <none> 42h v1.34.4
root@master:~# kubectl get pods
NAME READY STATUS RESTARTS AGE
apache-pod 1/1 Running 0 40m
nginx-pod 1/1 Running 0 9m41s
root@master:~# kubectl get pods -o wide
NAME READY STATUS RESTARTS AGE IP NODE NOMINATED NODE READINESS GATES
apache-pod 1/1 Running 0 40m 10.10.226.82 worker-1 <none> <none>
nginx-pod 1/1 Running 0 9m47s 10.10.226.83 worker-1 <none> <none>
root@master:~# curl http://172.31.46.47:30800
curl: (7) Failed to connect to 172.31.46.47 port 30800 after 0 ms: Couldn't connect to server
root@master:~# kubectl get svc
NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE
apache-service ClusterIP 10.99.14.121 <none> 80/TCP 42m
kubernetes ClusterIP 10.96.0.1 <none> 443/TCP 50m
nginx-nodeport NodePort 10.106.95.89 <none> 80:30080/TCP 11m
root@master:~# curl http://172.31.46.47:30080
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>

```

Task 3: Deploy a ReplicationController to maintain 3 replicas of nginx pod

Step 1: Create RC YAML

vi nginx-rc.yaml

Add:

apiVersion: v1

kind: ReplicationController

metadata:

name: nginx-rc

spec:

replicas: 3

selector:

app: nginxrc

template:

metadata:

labels:

app: nginxrc

spec:

containers:

- name: nginx

image: nginx

ports:

- containerPort: 80

Explanation: ReplicationController maintains 3 nginx pods always running.

```
root@master:~# vi nginx-rc.yaml
root@master:~# cat nginx-rc.yaml
apiVersion: v1
kind: ReplicationController
metadata:
  name: nginx-rc
spec:
  replicas: 3
  selector:
    app: nginxrc
  template:
    metadata:
      labels:
        app: nginxrc
    spec:
      containers:
      - name: nginx
        image: nginx
        ports:
        - containerPort: 80
```

Step 2: Apply RC

kubectl apply -f nginx-rc.yaml

Step 3: Verify RC

kubectl get rc

```
root@master:~# kubectl apply -f nginx-rc.yaml
replicationcontroller/nginx-rc created
root@master:~# kubectl get rc
NAME         DESIRED   CURRENT   READY   AGE
nginx-rc     3         3         3       24s
root@master:~#
```

Step 4: Verify pods

kubectl get pods

Explanation: RC creates pods automatically using template.

```
root@master:~# kubectl get pods
NAME                READY   STATUS    RESTARTS   AGE
nginx-rc-8jflx      1/1     Running   0          65s
nginx-rc-prkfl      1/1     Running   0          65s
nginx-rc-skngf      1/1     Running   0          65s
root@master:~#
```

Task 4: Scale the ReplicationController from 3 to 5 replicas

Step 1: Scale RC

kubectl scale rc nginx-rc --replicas=5 or we can go to yml file and change it there as well.

Explanation: Scaling changes the number of pods maintained by RC.

```
root@master:~# kubectl scale rc nginx-rc --replicas=5
replicationcontroller/nginx-rc scaled
root@master:~# kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
nginx-rc-8jflx	1/1	Running	0	7m33s
nginx-rc-bq2q4	1/1	Running	0	21s
nginx-rc-j7768	1/1	Running	0	21s
nginx-rc-prkfl	1/1	Running	0	7m33s
nginx-rc-skngf	1/1	Running	0	7m33s

Step 2: Verify pods count

kubectl get pods

Step 3: Verify RC count

kubectl get rc

```
root@master:~# kubectl get rc
```

NAME	DESIRED	CURRENT	READY	AGE
nginx-rc	5	5	5	9m5s

Task 5: Create a ReplicaSet to manage pods based on multiple labels (prod and test)

Step 1: Create ReplicaSet YAML

vi multi-label-rs.yaml

Add:

apiVersion: apps/v1

kind: ReplicaSet

metadata:

name: multi-label-rs

spec:

replicas: 3

selector:

matchLabels:

app: multiapp

env: prod

template:

metadata:

labels:

app: multiapp

env: prod

spec:

containers:

- name: nginx

image: nginx

Explanation: ReplicaSet uses multiple labels to select and manage pods.

```
root@master:~# vi multi-label-rs.yaml
root@master:~# cat multi-label-rs.yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: multi-label-rs
spec:
  replicas: 3
  selector:
    matchLabels:
      app: multiapp
      env: prod
  template:
    metadata:
      labels:
        app: multiapp
        env: prod
    spec:
      containers:
        - name: nginx
          image: nginx
root@master:~# |
```

Step 2: Apply ReplicaSet

kubectl apply -f multi-label-rs.yaml

```
root@master:~# kubectl apply -f multi-label-rs.yaml
replicaset.apps/multi-label-rs created
root@master:~# kubectl get pods
NAME                READY   STATUS    RESTARTS   AGE
multi-label-rs-7h7xd 1/1     Running   0           9s
multi-label-rs-w47q7 1/1     Running   0           9s
multi-label-rs-z6mj6 1/1     Running   0           9s
root@master:~#
```

Step 3: Verify ReplicaSet

`kubectl get rs`

Step 4: Verify pods with labels

`kubectl get pods --show-labels`

```
root@master:~# kubectl get rs
NAME                DESIRED   CURRENT   READY   AGE
multi-label-rs      3          3         3       53s
root@master:~# kubectl get pods --show-labels
NAME                READY   STATUS    RESTARTS   AGE   LABELS
multi-label-rs-7h7xd 1/1     Running   0           89s   app=multiapp,env=prod
multi-label-rs-w47q7 1/1     Running   0           89s   app=multiapp,env=prod
multi-label-rs-z6mj6 1/1     Running   0           89s   app=multiapp,env=prod
root@master:~#
```

Task 6: Deploy a ReplicaSet that excludes pods with label backend

Step 1: Create ReplicaSet YAML with matchExpressions

`vi exclude-backend-rs.yaml`

Add:

`apiVersion: apps/v1`

`kind: ReplicaSet`

`metadata:`

`name: exclude-backend-rs`

`spec:`

`replicas: 2`

`selector:`

`matchExpressions:`

`- key: role`

`operator: NotIn`

values:

- backend

template:

metadata:

labels:

role: frontend

spec:

containers:

- name: nginx

image: nginx

Explanation: matchExpressions allows advanced selection like excluding specific labels.

```
root@master:~# vi exclude-backend-rs.yaml
root@master:~# cat exclude-backend-rs.yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: exclude-backend-rs
spec:
  replicas: 2
  selector:
    matchExpressions:
      - key: role
        operator: NotIn
        values:
          - backend
  template:
    metadata:
      labels:
        role: frontend
    spec:
      containers:
        - name: nginx
          image: nginx
```

Step 2: Apply ReplicaSet

kubectl apply -f exclude-backend-rs.yaml

```
root@master:~# kubectl apply -f exclude-backend-rs.yaml
replicaset.apps/exclude-backend-rs created
root@master:~# kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
exclude-backend-rs-9tqvz            1/1     Running   0           8s
exclude-backend-rs-czrmw            1/1     Running   0           8s
multi-label-rs-2cfjc                1/1     Running   0          2m21s
multi-label-rs-gdxqx                1/1     Running   0          2m21s
multi-label-rs-rqn9m                1/1     Running   0          2m21s
root@master:~#
```

Step 3: Verify ReplicaSet and pods

kubectl get rs

kubectl get pods --show-labels

```

root@master:~# kubectl get pods --show-labels
NAME                                DESIRED    CURRENT    READY    AGE
exclude-backend-rs                  2          2          2        91s
multi-label-rs                       3          3          3        15m
NAME                                READY     STATUS    RESTARTS   AGE    LABELS
exclude-backend-rs-9tqvz            1/1      Running   0           91s    role=frontend
exclude-backend-rs-czrmw            1/1      Running   0           91s    role=frontend
multi-label-rs-2cfjc                1/1      Running   0          3m44s  app=multiapp,env=prod
multi-label-rs-gdxqx                1/1      Running   0          3m44s  app=multiapp,env=prod
multi-label-rs-rqn9m                1/1      Running   0          3m44s  app=multiapp,env=prod
root@master:~#

```

Task 7: Test load balancing across multiple pods using NodePort service

Step 1: Create deployment with 3 replicas

kubectl create deployment lb-nginx --image=nginx --replicas=3

Explanation: Deployment creates ReplicaSet which maintains 3 pods.

```

root@master:~# kubectl create deployment lb-nginx --image=nginx --replicas=3
deployment.apps/lb-nginx created
root@master:~# kubectl get pods
NAME                                READY     STATUS    RESTARTS   AGE
lb-nginx-cd98c8687-bp8wm            1/1      Running   0           13s
lb-nginx-cd98c8687-dsk7v            1/1      Running   0           13s
lb-nginx-cd98c8687-sg6c7            1/1      Running   0           13s
root@master:~#

```

Step 2: Expose deployment using NodePort

kubectl expose deployment lb-nginx --type=NodePort --port=80

Explanation: Service will load balance traffic across all pods.

```

root@master:~# kubectl expose deployment lb-nginx --type=NodePort --port=80
service/lb-nginx exposed
root@master:~# kubectl get pods
NAME                                READY     STATUS    RESTARTS   AGE
lb-nginx-cd98c8687-bp8wm            1/1      Running   0           82s
lb-nginx-cd98c8687-dsk7v            1/1      Running   0           82s
lb-nginx-cd98c8687-sg6c7            1/1      Running   0           82s
root@master:~#

```

Step 3: Check service and nodeport

kubectl get svc lb-nginx

Step 4: Get node IP

kubectl get nodes -o wide

```

root@master:~# kubectl get pods
NAME                                READY     STATUS    RESTARTS   AGE
lb-nginx-cd98c8687-bp8wm            1/1      Running   0           82s
lb-nginx-cd98c8687-dsk7v            1/1      Running   0           82s
lb-nginx-cd98c8687-sg6c7            1/1      Running   0           82s
root@master:~# kubectl get svc lb-nginx
NAME      TYPE      CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE
lb-nginx  NodePort  10.104.148.209  <none>           80:31510/TCP     50s
root@master:~# kubectl get nodes -o wide
NAME      STATUS    ROLES    AGE    VERSION    INTERNAL-IP    EXTERNAL-IP    OS-IMAGE             KERNEL-VERSION    CONTAINER-RUNTIME
master    Ready     control-plane  45h   v1.34.4    172.31.32.135  <none>         Ubuntu 24.04.3 LTS   6.14.0-1018-aws   containerd://1.7.28
worker-1  Ready     <none>      43h   v1.34.4    172.31.46.47  <none>         Ubuntu 24.04.3 LTS   6.14.0-1018-aws   containerd://1.7.28
worker-2  Ready     <none>      43h   v1.34.4    172.31.46.88  <none>         Ubuntu 24.04.3 LTS   6.14.0-1018-aws   containerd://1.7.28

```

Step 5: Test using curl multiple times

curl http://<NODE-IP>:<NODEPORT>

```
lb-nginx-cd98c8687-sg6c7 1/1 Running 0 13s
root@master:~# kubectl expose deployment lb-nginx --type=NodePort --port=80
service/lb-nginx exposed
root@master:~# kubectl get pods
NAME READY STATUS RESTARTS AGE
lb-nginx-cd98c8687-bp8wm 1/1 Running 0 82s
lb-nginx-cd98c8687-dsk7v 1/1 Running 0 82s
lb-nginx-cd98c8687-sg6c7 1/1 Running 0 82s
root@master:~# kubectl get svc lb-nginx
NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE
lb-nginx NodePort 10.104.148.209 <none> 80:31510/TCP 50s
root@master:~# kubectl get nodes -o wide
NAME STATUS ROLES AGE VERSION INTERNAL-IP EXTERNAL-IP OS-IMAGE KERNEL-VERSION CONTAINER-RUNTIME
master Ready control-plane 45h v1.34.4 172.31.32.135 <none> Ubuntu 24.04.3 LTS 6.14.0-1018-aws containerd://1.7.28
worker-1 Ready <none> 43h v1.34.4 172.31.46.47 <none> Ubuntu 24.04.3 LTS 6.14.0-1018-aws containerd://1.7.28
worker-2 Ready <none> 43h v1.34.4 172.31.46.88 <none> Ubuntu 24.04.3 LTS 6.14.0-1018-aws containerd://1.7.28
root@master:~# curl http://172.31.46.47:31510
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
root@master:~#
```

curl http://<NODE-IP>:<NODEPORT>

```
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
root@master:~# curl http://172.31.46.47:31510
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
root@master:~#
```

curl http://<NODE-IP>:<NODEPORT>

```
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
root@master:~# curl http://172.31.46.88:31510
<!DOCTYPE html>
<html>
<head>
<title>welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto; }
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
```

Explanation: Service sends requests to different pods, but nginx output looks same. To confirm load balancing, pod hostname can be shown later.

Task 8: Delete ReplicationController without affecting running pods

Step 1: Delete RC with orphan option

→ create one rc file and run it

```
root@master:~# vi nginx-rc.yml
root@master:~# cat nginx-rc.yml
apiVersion: v1
kind: ReplicationController
metadata:
  name: nginx-rc
spec:
  replicas: 3
  selector:
    app: nginxrc
  template:
    metadata:
      labels:
        app: nginxrc
    spec:
      containers:
      - name: nginx
        image: nginx
        ports:
        - containerPort: 80
root@master:~# |
```

→ lets delete rc without deleting pods

```
root@master:~# kubectl apply -f nginx-rc.yml
replicationcontroller/nginx-rc created
root@master:~# kubectl get pods
NAME                                READY    STATUS    RESTARTS   AGE
1b-nginx-cd98c8687-c997n            1/1      Running   0           3m41s
1b-nginx-cd98c8687-jkjm7            1/1      Running   0           3m41s
1b-nginx-cd98c8687-qpzcx            1/1      Running   0           3m41s
nginx-rc-sbtsh                      1/1      Running   0           10s
nginx-rc-wzr9s                      1/1      Running   0           10s
nginx-rc-z8wnc                      1/1      Running   0           10s
```

kubectl delete rc nginx-rc --cascade=orphan

Explanation: Orphan option deletes controller but keeps pods running.

Step 2: Verify RC is deleted

kubectl get rc

```
replicationcontroller/nginx-rc deleted from default namespace
root@master:~# kubectl get rc
No resources found in default namespace.
root@master:~#
```

Step 3: Verify pods still running

kubectl get pods

```
root@master:~# kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
lb-nginx-cd98c8687-c997n            1/1     Running   0           6m39s
lb-nginx-cd98c8687-jkjm7            1/1     Running   0           6m39s
lb-nginx-cd98c8687-gpzcx            1/1     Running   0           6m39s
nginx-rc-sbtsh                       1/1     Running   0           3m8s
nginx-rc-wzr9s                      1/1     Running   0           3m8s
nginx-rc-z8wnc                      1/1     Running   0           3m8s
root@master:~#
```

Explanation: Pods remain but are unmanaged, so scaling will not work now.

Conclusion

After completing all the tasks, I have gained clear understanding of how Pods and Containers work, how Services (ClusterIP and NodePort) provide stable communication and external access, and how controllers like ReplicationController and ReplicaSet maintain the desired number of pods. I also understood how scaling works, how labels and selectors connect resources, and how to verify everything using Kubernetes troubleshooting commands. Overall, this practice gives me a strong foundation to move forward into advanced Kubernetes topics like Deployments, ConfigMaps, Secrets, and Ingress.